StepOverflow: PMC Overflows on AMD SEV

Anja Rabich
University of Luebeck
Lübeck, Germany
a.rabich@uni-luebeck.de

Jonah Heller
University of Luebeck
Lübeck, Germany
j.heller@uni-luebeck.de

Florian Sieck
University of Luebeck
Lübeck, Germany
florian.sieck@uni-luebeck.de

Thomas Eisenbarth
University of Luebeck
Lübeck, Germany
thomas.eisenbarth@uni-luebeck.de

Abstract

Trusted Execution Environments (TEEs) have seen widespread adoption in cloud computing for ensuring confidentiality and integrity, even against the cloud provider. However, research has shown that TEEs are vulnerable to side-channel attacks.

In this work, we demonstrate and evaluate a side-channel that exploits Performance Monitoring Counter (PMC) overflows as an event-based interrupt primitive for AMD SEV-SNP. We show that the control-flow of workloads running in SEV VMs can be reconstructed from the information leaked by PMC overflows. A latency inherent to PMC overflows, called skid, prevents the interrupt primitive from achieving instruction-level granularity. Although the evaluation on an AMD Zen 3 CPU reveals significant skid, we construct a single-trace instruction counting attack with our primitive. In a case study on TinyCrypt, we evaluate the ability of PMC overflows to leak secret bits of the nonce k from the modular inverse function of an ECDSA signature scheme. We find that, with limitations, sufficient bits of k can be recovered.

CCS Concepts: • **Security and privacy** → **Trusted computing**; **Cryptanalysis and other attacks**.

Keywords: Trusted execution; virtualization; SEV; side-channel

ACM Reference Format:
Anja Rabich, Jonah Heller, Florian Sieck, and Thomas Eisenbarth. 2026. StepOverflow: PMC Overflows on AMD SEV. In *9th Workshop on System Software for Trusted Execution (SysTEX '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3805690.3805727

This work is licensed under a Creative Commons Attribution 4.0 International License.
SysTEX '26, Edinburgh, Scotland Uk
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2607-1/26/04
https://doi.org/10.1145/3805690.3805727

1 Introduction

TEEs have emerged as a critical security mechanism in cloud computing, providing customers with integrity and confidentiality guarantees for executing sensitive workloads, and without the need to trust the cloud vendor. By isolating computations within hardware-protected enclaves or confidential virtual machine (CVM), TEEs such as Intel Software Guard Extensions (SGX), AMD Secure Encrypted Virtualization (SEV), and Intel Trust Domain Extensions (TDX) aim to prevent unauthorized access to data and code, regardless of the underlying hardware and software stack.

Previous work has demonstrated that TEEs are vulnerable to a range of side-channel attacks. Interrupt-based attacks [18, 19], in particular single-stepping [6, 17, 24, 25], are especially powerful. These leverage host-to-guest context switches to leak or can even manipulate control-flow within the TEE.

Van Bulck et al. implement the first framework for instruction-level single-stepping of SGX enclaves [6] using Advanced Programmable Interrupt Controller (APIC) timer interrupts. Wilke et al. apply the same technique to AMD SEV in SEV-Step [25], while Gast et al. [10] extend SEV-Step with additional Performance Monitoring Counter (PMC) events. Although this technique achieves maximum temporal resolution, using the APIC timer to pause TEEs after every instruction leads to significant execution slowdown.

In this work, we investigate the feasibility of using PMC overflow interrupts on AMD SEV Secure Nested Paging (SEV-SNP) as an attack primitive without APIC timer interrupts. PMCs can be configured to trigger an interrupt when their counter register overflows. By setting the initial event counter value to the overflow value minus the number of events to be observed, interrupts can be triggered in a targeted manner. After the overflow, the PMC register can be reset in a custom interrupt handler [1]. The configuration of and access to PMCs, cannot be prevented by SEV-SNP guests on AMD Zen 1-4 CPUs [10].

Previous research on PMCs either consider a different attacker model [21] than the common TEE attacker model, do not use PMC overflow interrupts [14] or do not demonstrate this technique on AMD CPUs [20].

We aim to answer the following questions:

RQ1. Can we use PMC overflow interrupts to achieve the same temporal resolution as single-stepping?

RQ2. Can PMC overflow interrupts also be used on SEV-SNP?

RQ3. Can PMC overflow interrupts be used as an attack primitive for reconstructing confidential control-flow?

Contributions. In summary, we contribute the following:

- We provide an analysis of PMCs overflow interrupts and their technical challenges on an AMD Zen 3 CPU.
- We apply the use of PMCs overflow interrupts on targets running in SEV-SNP for information leakage.
- In a case study on TinyCrypt, we evaluate the use of PMC overflow interrupts for a single-trace attack that leaks bits in non-constant time code.

2 Background

2.1 Trusted Execution Environment (TEE)

Trusted Execution Environments isolate compartments for secure computations on untrusted hardware. Under this threat model, only the CPU is deemed trustworthy, while all other components and the Operating System (OS) are untrusted. TEE designs differ in the design of their protected domains. While Intel SGX creates compartments for userland processes, other designs such as AMD SEV, Intel TDX, and ARM CCA protect full virtual machines, often referred to as confidential virtual machines (CVMs). These encrypt memory for data at rest, thus preventing direct insight by the untrusted components. For AMD SEV, the current Virtual Machine (VM) state, including memory and registers, is stored encrypted in the virtual machine save area (VMSA) during context switches between guest and hypervisor. SEV-SNP offers the highest form of protection by also providing a shadow mapping of VM-page ownership to prevent replay and remapping attacks.

2.2 Performance Monitoring Counter

CPU manufacturers introduced Performance Monitoring Counters (PMCs) for profiling software execution events and resource usage. By tracking the frequency of specific events or CPU cycles spent on a particular task, developers can identify performance bottlenecks and optimize code. On x86, these counters are configured and accessed via dedicated Model Specific Registers (MSRs).

Both AMD and Intel have made a wide range of performance counter events available, although the specific events differ. The total number of simultaneously observable events is specific to the CPU model. Events can also trigger an interrupt whenever a counter overflows. An interrupt handler can then be programmed to reset the counter value, and re-initiate a new sampling period [20]. On AMD, events can also be restricted to count host or guest events or both, as well as events from kernel space or userland or both during execution.

Skid. One challenge when relying on PMC overflows to detect events is *skid*. Skid refers to the deviation of the instruction pointer (IP) from the instruction that triggers a PMC overflow, and the attributed instruction in the interrupt routine that handles the overflow [9]. The latter is the interrupt handler's return address. This offset can also be viewed as a delay between triggering and reporting an event. The delay is primarily caused by out-of-order execution and influenced by the Re-order Buffer (ROB) size [8, 9]. This results in the reported IP being several instructions away from the actual instruction that triggered the event, leading to inaccurate performance profiling. AMD Instruction-Based Sampling (IBS) [9] and Intel Precise Event-Based Sampling (PEBS) [13] were introduced to achieve more precise event attribution. As side-channel protection, SEV-SNP guests can prevent the hypervisor from using IBS to gather information on their execution [1].

Security Measures. To mitigate the risk of leaking sensitive information on workloads, Intel SGX and TDX restrict direct access to PMCs unless the TEE is in debug mode. However, AMD SEV does not prevent the host from programming and reading counter events of the guest for CPU architectures Zen 1-4. Zen 5 introduces a new MSR configuration bit, allowing SEV guests to disable hypervisor access to their counter values [10].

2.3 Threat Model

We operate under the standard threat model for TEEs and assume the adversary has full control over the hypervisor and the OS. The adversary can configure hardware interrupts and PMCs at will. While the OS is responsible for managing hardware and memory resources, only the CPU is considered part of the TCB. We also assume that the attacker can observe and manipulate Page Table Entries (PTEs). This would allow them to mount a page-based controlled-channel to observe course-grained control-flow behavior of a SEV guest.

3 Related Work

We summarize previous work on interrupt-based TEE side-channels and applications of PMC overflows on virtualization.

3.1 Interrupt-Based TEE Side-Channels

Van Bulck et al. implement SGX-Step [6], which triggers interrupts at deterministic timer intervals to single-step the execution of SGX enclaves. By setting the APIC timer in one-shot mode with precise timing calibration directly from user space, SGX-Step achieves deterministic, instruction-level single-stepping. To interrupt during the enclave's first

instruction, SGX-Step stretches the first instruction's execution time by removing the "accessed" (A) bit on the enclave's Page Middle Directory (PMD) which triggers a microcode-assisted page-table walk [7].

Wilke et al. introduce SEV-Step [25], a Linux kernel framework for implementing single-stepping and page-based controlled-channels on AMD SEV-SNP. For single-stepping, the APIC timer is also programmed in one-shot mode. To confirm the retirement of the first instruction after resuming the CVM, the framework uses the 'dirty' (D) bit in the PTE of the code page. The Translation Lookaside Buffer (TLB) of the guest CVM is flushed prior to each entry, to slow down the first instruction. Single- and multi-steps are distinguished with the information from the retired instruction performance counter.

Gast et al. [10] further extend SEV-Step with additional PMC events such as branching information. Combined with instruction-level precision, this allows for leaking secret-dependent control-flow. They do not explore the use of PMC overflows for interrupting TEE execution.

3.2 Applications of PMC Overflows on Virtualization

Runtime Attestation. Vogl and Eckert [21] demonstrate the use of PMCs on Intel CPUs for instruction-level monitoring of VMs to detect control-flow hijacking. Their approach leverages PMCs to monitor specific instruction types, such as branch instructions. The hypervisor configures the corresponding counters to generate a Non-Maskable Interrupt (NMI) upon overflow, notifying it of events of interest. Thereby, the hypervisor can reconstruct the guest's execution with high precision. To verify return addresses, they implement an evasion-resistant shadow stack. To address missed instructions due to skid, Vogl and Eckert use the Last Branch Record (LBR) stack to reconstruct the execution path, ensuring accurate attestation even when multiple instructions are executed between interrupts.

Attack Primitives. Spisak uses PMC overflows to intercept syscalls and implements a rootkit for ARM and x86 [20]. On x86, the rootkit uses PMCs to trap far branches, which serve as a clear signature for the `SYSCALL` instruction. Malicious code can then be inserted into the interrupt handler. Lou et al. [14] show how PMCs can be used to fingerprint applications running in TEEs. As a mitigation, they provide a framework that identifies, then obfuscates PMC values using additional gadgets. They do not explore the use of PMC overflows for interrupting TEE execution.

4 Setup

Our experiments were performed on an AMD EPYC 7763 64-Core Processor (Zen 3). The host and SEV-SNP VM both run Ubuntu 22.04.4 LTS. For the host, we use the Linux kernel 6.16.0 and qemu from the AMDSEV git repository on branch `snp-latest` (commit `69a015ff`). The CPU frequency is fixed to 2.4GHz. For the experiment in Section 5, we isolate and pin the core for our target application. For Section 6, we isolate and pin the core used by our VM. In our case study in Section 7, we reduce noise caused by other VM user space processes by additionally isolating a dedicated vCPU in the guest and scheduling the target on this isolated core. Strictly speaking, this is not possible in the common TEE attacker model. However, for analysis purposes of the attack primitive, we opt for cleaner traces and argue that the additional noise could be filtered out by tracking all CVM pages and only enable the PMC overflow primitive on the target page.

5 Harnessing PMC Overflows

To answer **RQ1**, we design an experiment for evaluating the temporal granularity of interrupts using PMC overflows in a non-protected, non-virtualized host environment. Since skid induces latency in PMC measurements and event interrupts, we analyze its influence on the temporal resolution of the PMC-based interrupt primitive. To the best of our knowledge, previous work has implemented PMC overflows on Intel CPUs [20, 21], with no information on other x86 platforms.

Measurement Setup. We implement a kernel module with an NMI interrupt handler for PMC overflows and for configuring PMCs. The module's interrupt handler also logs the programmed PMC values. The overflow counter is initialized with the maximum integer value minus the desired sampling interval. Whenever an overflow occurs, the interrupt handler resets all performance counter values for the next sample. On every interrupt, our handler resets all performance counter values and logs the current return `IP`. We configure all PMCs to only count host userland events for experiments on the host.

5.1 Skid

Experiment Design. Listing 1 shows the experiment we use to evaluate the impact of the skid on performance counter values. We configure a PMC interrupt on every `AVX` *Retired MMX/FP Instructions* event. To evaluate the impact of different instruction types on the skid, we repeat the experiments several times. Therefore, for each repetition, `$i$` is replaced with a slide of another instruction type. We use instructions with different latencies, as specified for our Zen 3 CPU on uops.info. By comparing the `IP`s logged by our overflow interrupt handler with the address of the `vpxor` instructions in our binary, we can calculate the total number of test slide instructions that were executed. After the second `vpxor` instruction, we append an `add`-slide for all experiments. This allows us to compare the different `$i$`-instructions. Additionally, it also allows for resetting other PMC events that were configured for `$i$`.

Results. When `$i$` is set to the `div` instruction, we observe that an average of 105 instructions are executed. For

```
.loop                          // repeat N times
    lfence
    vpxor xmm0, xmm1, xmm0     // overflow (start)
    .rept 4000
        $i$                    // test slide
    .endr

    lfence
    vpxor xmm0, xmm1, xmm0     // overflow (stop)
    .rept 4000
        add r13, r13           // static slide
    .endr
    jnz loop
```

Listing 1. Pseudocode for measuring PMC skid, by triggering two PMC overflows. An instruction (`$i$`) is repeated 4000 times to ensure that each overflow is handled before the next trigger is executed. The test slide is used to pinpoint the number of instructions executed due to skid. A static slide is executed in between all measurements, to compare between different test slides and for resetting all PMC events before executing the test slide again. Using the `IPs` logged by our interrupt handler the exact number of executed instruction between overflows can be calculated which stem from skid.

this particular instruction slide, we configure another PMC to count the total number of executed `div` instructions. We confirm that the `div` instructions counted by the PMC match the calculated number of instructions using the instruction `IPs`. Hence, we can conclude that our collected PMC values are accurate when measuring the number of executed instructions between overflow events.

Using an add as the `$i$`-instruction, we observe an average of 2043 instructions executed in the test slide. The same number is observed for the static slide. Although the number of instructions executed between interrupts becomes less with increasing instruction latency, our temporal resolution is nonetheless limited. Table 1 in the appendix lists the skid for all evaluated instructions.

We observe for several loop-iterations, an event count for retired `vpxor` instructions that is greater than the configured overflow. Hence, we know that both overflows from Listing 1 were combined into one interrupt. The total number of counted interrupts is also less than the number of loop-iterations (100,000).

We conclude that the PMC skid imposes restrictions on the granularity of the observable information but is still deterministic.

> **RQ 1** Latency due to PMC skid is too great to achieve instruction-level interrupts using PMC overflows.

```
void func(){
    int inptr[] = {0, 0, 0, 0, 1, 0, 0, 1};
    int x = 0;
    for(int ctr = 0; ctr < 8; ctr++) {
        if (inptr[ctr] == 0) {
            //random instructions so that
            //if is not optimized out
            x += 23;
            x *= 1337;
        }

        //padding asm slide
        .rept 4000
        add r13, r13
        .endr
    }
}
```

Listing 2. Pseudocode of our function for testing control-flow extraction from the host and SEV CVM. We add a padding add-slide after the if-branch, such that the PMC overflow handler reliably triggers on a conditional jump.

5.2 Control-Flow Reconstruction

As a prelude to answering **RQ3**, we evaluate the feasibility of control-flow reconstruction with PMC overflows in a non-confidential, non-virtualized environment on AMD.

Experiment Design. In order to test whether control-flow can be inferred with PMC overflow interrupts, we construct a non-constant time test case. As shown in Listing 2, an if-statement is evaluated in each loop-iteration based on the current array input. The if-statement is populated with several low-latency dummy instructions to prevent their removal by compiler optimizations. After the input-dependent if-statement, we append an add-slide to ensure that the PMC interrupt triggers on every loop iteration. We configure the PMCs to only count host userland events and overflow on *Retired Conditional Branch* events while also collecting PMC values for *Retired Taken Branch Instructions*.

Results. With our collected PMC values, we are able to reconstruct the array content from the input-dependent if-statement.

6 Harnessing PMC Overflows in SEV-SNP

Having confirmed that we can use PMC overflows for event-based interrupts, we turn our attention to **RQ2**. We strive to evaluate whether the results from Section 5 are applicable to AMD SEV-SNP and whether the skid differs from a non-protected non-virtualized environment.

As described in Section 2.2, PMC events of SEV guests can be accessed by the hypervisor for Zen 1-4. However, the IP cannot be accessed by the host unless the VM is in debug mode and the IP is decrypted from the VMSA. In addtion, there is the potential for noise due to context switching between host and guest or within the guest, which could also

impact the temporal resolution when using PMC overflow interrupts.

6.1 Skid

Experiment. To measure the skid in SEV VMs, we rerun the experiment from Listing 1 in our SEV-SNP guest, while collecting PMC values using our kernel module. Since we cannot count the total number of executed instructions using the IP, we use `div` instructions in Listing 1, because the total number of executed instruction can be directly counted using the *Div Op Count* event. Hence, calibration can be performed entirely from the PMC values that are visible to the hypervisor.

Results. We observe no difference in skid compared to experiments performed on the host in Section 5.

6.2 Trace Noise

We also repeat the experiment from Listing 2 in AMD SEV-SNP. Since we cannot observe the IPs, we place a `clflush` instruction in the main function prior to calling the measurement function. This can be directly observed using a corresponding PMC event. The end of the measurement function is observed through the *Retired Near Returns* event.

Results. We are able to reconstruct the array's contents from the input-dependent if-statement. However, our collected logs contain entries that do not belong to our target. To investigate this noise, we set two PMC registers to count retired instructions. One only counts guest user space events while the other only counts kernel space events. While executing Listing 2, we observe that the guest kernel regularly executes instructions throughout the trace. We conclude that the guest kernel schedules other user processes in between the target's execution, which are then visible as noise in our user space PMC counts.

> **RQ 2** PMC overflows can be used on SEV-SNP. The skid does not change in this environment.

7 Analyzing modInv in TinyCrypt's ECDSA

In the previous section, we determined that leaking information from victim applications and interrupting them with PMC overflows can be applied to targets in SEV-SNP.

In this section, we evaluate whether the temporal and spatial resolution of the PMC overflow side-channel is sufficient to attack real-world cryptographic implementations. As a case study, we evaluate how PMC overflows can be used to leak the least significant bits of an input to the modular inversion of TinyCrypt's ECDSA implementation. This could be used to leak the ephemeral secret k.

```
1 void uECC_vli_modInv(uECC_word_t *result,
2   uECC_word_t *input, uECC_word_t *mod
3   wordcount_t num_words){
4   cmpresult_t cmpResult;
5
6   while ((cmpResult = uECC_vli_cmp_unsafe(
7         input, mod, num_words)) != 0) {
8     if (EVEN(input)) {        //LSB == 0
9       // shifts input right by one bit
10    } else if (EVEN(mod)) {   //LSB == 0
11      // shifts mod right by one bit
12    } else if (cmpResult > 0) {
13      // ...
14    } else {
15      // ...
16    }
17  }
18 }
```

Listing 3. Shortened and simplified listing of the modular invert function from TinyCrypt [12]. In the attack case study, the function is observed for inverting the ephemeral ECDSA secret k. The first two `if`-cases are very short with few instructions, the last two cases are significantly longer.

TinyCrypt modInv. TinyCrypt [12] is a small cryptographic library that supports several cryptographic primitives, including the signature algorithm ECDSA. TinyCrypt includes only the curve secp256r1. For our evaluation, we chose TinyCrypt for its small memory footprint and its non-constant implementation of modular inversion. This is used in the ECDSA signing process to invert the ephemeral secret k. Explaining the ECDSA signing process and its weaknesses, is beyond the scope of this paper. For the following analysis, it is important that signing with ECDSA requires a nonce k that must be unpredictable by the attacker, must not be repeated for different messages, and must be kept secret. Leaking only a few bits of k and recording several message signature pairs together with the leaked bits of k suffices to reconstruct the long term private ECDSA key by solving the hidden number problem with lattice [5, 11, 16, 22] or Fourier analysis-based methods [2, 4, 15]. For our evaluation, we directly call TinyCrypt's modular inversion function `uECC_vli_modInv()`. The code is shown in Listing 3.

TinyCrypt already offers the option to enable masking k to mitigate the timing vulnerability. For the purposes of our analysis, we leave masking disabled. The function computes the modular inverse in a `while`-loop that executes one of four different if-cases per iteration. The first case (line 9) checks whether the input is even, meaning zero in the least significant bit. If yes, it updates the modular inverse calculation and right shifts the input by one bit for the next iteration. The second case (line 11) does the same for the modulus, which, for ECDSA, must be a prime number. If the first case is taken, an attacker learns the least significant bit of the input. Hence, if the first n iterations of the `while`-loop

enter the first case, the attacker learns that the n least significant bits of k are zero. Both cases execute a relatively small number of instructions. The last two cases also operate on the input and modulus and update the modular inverse calculation, but execute other instructions, leaving a different footprint in the counted PMC values. Thus, they can be distinguished from the first two cases.

Since the modulus must be prime, the second case can only be entered after the fourth case was entered and the modulus was modified. Thus, the attacker can be sure that before they observe the third or fourth `if`-case, all previous iterations executed the first case.

Experiment Design and Measurement Setup. We only build the necessary translation unit `ecc.c` from TinyCrypt with gcc's `-Os` build parameter and link it against our small main application. The resulting binary fits into one memory page. Analyzing the non-constant time behavior with a page-fault controlled-channel is thus not possible. To further slow down execution of the CVM, we invalidate both the TLB and cache during PMC overflow interrupt handling.

As in the previous section, the target is started in an AMD SEV-SNP CVM and the PMC overflows are programmed and observed from the host. We program the counter registers to observe the events *Retired Taken Branch Instructions*, *Conditional Branch Instructions*, *Retired Instructions* and *Retired Near Returns* for user space processes.

In a usual attack scenario, the called function `uECC_vli_modInv()` would be on a different page than its calling function, while the function itself and all the functions it calls remain on the same page. In such a scenario, the analysis with the page-fault controlled-channel is not feasible and the single code page change would provide us with a defined entry point. Since our binary fully fits into a single page, we simulate the fixed entry point with a slide of 105 `div` instructions between line 13 and 14 of Listing 3. We set the PMC overflow on the *Retired Taken Branch Instructions* instructions counter. In the interrupt handler we retrieve the number of committed `div` instructions and thereby fix the starting point of our analysis. If we interrupt several times within the `div` slide, we can ignore these interrupts and subtract the number of `div` instructions from the PMC values of the first relevant interrupt. Subsequently, we continue to interrupt on the *Retired Taken Branch Instructions* counter and to collect the configured PMC values on each interrupt for the leakage analysis step.

Leakage Analysis. As described in the previous paragraph, we leak consecutive trailing zero bits of k from the `while`-loop in `uECC_vli_modInv()` by tracking the configured PMC events and regularly interrupting on the *Retired Taken Branch Instructions* overflow. Therefore, our goal is to exploit the collected PMC value traces to reconstruct the control-flow taken by the `uECC_vli_modInv()`, more specifically, to determine how many of the first consecutive iterations of the `while`-loop enter the first `if`-case.

To reconstruct this information from the PMC traces, we only need to identify how long the control-flow follows the path of entering the first `if`-case in every iteration, rather than completely reconstruct the control-flow. To this end, we collect simulated instruction-granular PMC values of the `while`-loop when only taking the first `if`-case and store them in a lookup table.

We model the impact of each instruction type in the target function's control-flow on the collected PMCs and execute the target function with gdb, configured to step every instruction. To simulate the intended control-flow path, we use an input with 32 leading least significant zero bits. For reconstructing the long-term ECDSA secret key of a 256-bit secp256r1 curve, knowledge of at least 3 to 4 consecutive bits of k are required, provided the attacker collects sufficient signature traces [3]. Leaking more bits reduces the number of required traces to solve the hidden number problem.

The lookup table serves as a hypothesis of the expected PMC values after every instruction, which we compare against the PMC values at the overflow sample points in the real collected traces. When the real trace deviates from the simulated trace, the attacker knows that the control-flow left the path corresponding to leading consecutive 0-bits.

Note that collecting the leakage on the k-values requires a single-trace recording of the PMC values for each signature, since k is randomly chosen and only used once. To reconstruct the long term ECDSA private key, the attacker would observe traces of ECDSA signature computations and collect those with trailing 0-bits in k, identified from the leakage. When the attacker has collected sufficient traces, they can use a lattice attack [3, 11, 16] to reconstruct the private key. The reconstruction is out of scope for this work.

Results. We test different inputs for k, trace the execution, and compare it against our simulated execution. Our evaluation indicates that we can correctly identify consecutive 0-bits and also give an approximation of the number of 0-bits. While our analysis is not always determining the correct number of consecutive least significant 0-bits, it only underapproximates the number by 0 to 5 bits. The errors are caused by missing overflow interrupts for some loop iterations.

RQ 3 PMC overflows can yield enough information to reconstruct confidential control-flow. However, this depends on the number of interrupts that can be observed and information gained from PMC events.

8 Discussion & Mitigations

We have demonstrated the use of PMC overflows for event-based sampling. This can also be applied to code run in SEV-SNP. By sampling at specific control-flow events such as taken branches and also collecting additional performance counter values, we are able to infer control-flow from SEV-SNP to some extent.

Limitations of PMC Overflows. The main limitation of PMC overflows as an attack primitive is its temporal resolution due to skid between the actual event and the overflow interrupt. This applies both to its use for runtime-attestation and as a side-channel. Although we were able recover bits of the nonce in TinyCrypt, this was only possible with modifications to the library itself. The primitive also relies on the current handling of PMC overflows.

Limitations of the Evaluation. Our experiments were conducted on a single machine, which leaves the behavior of PMC overflows on other AMD CPUs open. Other CPUs may show greater or smaller skid, as is indicated by previous work on Intel CPUs [21].

Comparison to Single-Stepping. Single-stepping with SEV-Step achieves the maximum possible temporal resolution, since it captures every executed instruction. This makes event attribution trivial and can thus be considered noiseless, while PMC overflows must take skid into account. Thus single-stepping achieves superior resolution. However, single-stepping comes at the cost of the execution throughput of the target. Since PMC overflows generate far fewer interrupts, they achieve an arguably higher throughput.

Improving Temporal Resolution. Since skid is influenced by the ROB size, we suspect that memory operands can reduce the total number of instructions required to overcome skid. To test the impact of memory delay on skid, we modify our experiment from Listing 1 to execute a `div` instruction with a memory operand. By flushing the memory address prior to the instruction, we can observe whether the number of executed instructions changes due to caching. We find that flushing the address reduces our measured skid from an average of `105` to `64` `div` instructions. Hence, we conclude that an attacker can delay memory operations and can thus reduce the number of required instructions for overcoming PMC skid. The exact measurements for this experiment are also listed in Table 1.

Mitigations. Zen 5 CPUs introduce configuration bits such that SEV guests can disallow host access to their PMC values [10]. In addition, PMC Virtualization may limit hypervisor visibility into guest counter values, which is also supported starting with Zen 5 [1]. However, PMC virtualization requires that virtualized interrupts are supported by the host and guest OS.

As a software mitigation, code can be implemented in constant time. For older processor generations, Weissteiner et al. propose TEEcorrelate [23], which decouples PMC values by aggregating them within a time window to reduce temporal resolution and adding some statistical deviation in the reported values. Lou et al. implement PMC obfuscation with additional instructions to noise PMC values [14].

9 Conclusion

We have analyzed the use of PMC overflows for event-based interrupts on an AMD Zen 3 CPU with SEV-SNP. Although PMC values are limited by an inherent latency that reduces their temporal resolution significantly, we find that collecting traces using overflow interrupts can still provide information for inferring control-flow of SEV-SNP guests. In a case study, we evaluate the use of PMC overflows for a single-trace attack on SEV-SNP.

10 Acknowledgments

This work has been supported by the BMFTR under the grant RILKOSAN and by the project *Vertrauenswürdiger Datenaustausch für verteilte Stromnetze (VeDS)* within the framework of the KI-Förderrichtlinie (AI funding guideline) by the State of Schleswig-Holstein.

References

[1] Advanced Micro Devices, Inc. 2024. *AMD64 Architecture Programmer's Manual.* Advanced Micro Devices, Inc. Available at: https://www.amd.com/en/support/tech-docs.

[2] Adi Akavia. 2009. Solving Hidden Number Problem with One Bit Oracle and Advice. In *Advances in Cryptology - CRYPTO 2009, 29th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2009. Proceedings (Lecture Notes in Computer Science, Vol. 5677)*, Shai Halevi (Ed.). Springer, 337–354. doi:10.1007/978-3-642-03356-8_20

[3] Martin R. Albrecht and Nadia Heninger. 2021. On Bounded Distance Decoding with Predicate: Breaking the "Lattice Barrier" for the Hidden Number Problem. In *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 12696)*, Anne Canteaut and François-Xavier Standaert (Eds.). Springer, 528–558. doi:10.1007/978-3-030-77870-5_19

[4] Daniel Bleichenbacher. 2000. On the generation of one-time keys in DL signature schemes. In *Presentation at IEEE P1363 working group meeting*. 81.

[5] Joachim Breitner and Nadia Heninger. 2019. Biased Nonce Sense: Lattice Attacks Against Weak ECDSA Signatures in Cryptocurrencies. In *Financial Cryptography and Data Security - 23rd International Conference, FC 2019, Frigate Bay, St. Kitts and Nevis, February 18-22, 2019, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 11598)*, Ian Goldberg and Tyler Moore (Eds.). Springer, 3–20. doi:10.1007/978-3-030-32101-7_1

[6] Jo Van Bulck, Frank Piessens, and Raoul Strackx. 2017. SGX-Step: A Practical Attack Framework for Precise Enclave Execution Control. In *Proceedings of the 2nd Workshop on System Software for Trusted Execution, SysTEX@SOSP 2017, Shanghai, China, October 28, 2017.* ACM, 4:1–4:6. doi:10.1145/3152701.3152706

[7] Scott Constable, Jo Van Bulck, Xiang Cheng, Yuan Xiao, Cedric Xing, Ilya Alexandrovich, Taesoo Kim, Frank Piessens, Mona Vij, and Mark Silberstein. 2023. AEX-Notify: Thwarting Precise Single-Stepping Attacks through Interrupt Awareness for Intel SGX Enclaves. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, 4051–4068. https://www.usenix.org/conference/usenixsecurity23/presentation/constable

[8] Sanjeev Das, Jan Werner, Manos Antonakakis, Michalis Polychronakis, and Fabian Monrose. 2019. SoK: The Challenges, Pitfalls, and Perils of Using Hardware Performance Counters for Security. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 20–38. doi:10.1109/SP.2019.00021

[9] Paul J. Drongowski. 2007. *Instruction-Based Sampling: A New Performance Analysis Technique for AMD Family 10h Processors*. Advanced Micro Devices, Inc. Available at: https://www.amd.com/content/dam/amd/en/documents/archived-tech-docs/white-papers/AMD_IBS_paper_EN.pdf.

[10] Stefan Gast, Hannes Weissteiner, Robin Leander Schröder, and Daniel Gruss. 2025. CounterSEVeillance: Performance-Counter Attacks on AMD SEV-SNP. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society. https://www.ndss-symposium.org/ndss-paper/counterseveillance-performance-counter-attacks-on-amd-sev-snp/

[11] Nick Howgrave-Graham and Nigel P. Smart. 2001. Lattice Attacks on Digital Signature Schemes. *Des. Codes Cryptogr.* 23, 3 (2001), 283–290.

[12] Intel. 2026. *TinyCrypt Cryptographic Library*. https://github.com/intel/tinycrypt

[13] Intel Corporation. 2025. *Intel® 64 and IA-32 Architectures Software Developer's Manual*. Intel Corporation. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html Volume 3B: System Programming Guide, Part 2.

[14] Xiaoxuan Lou, Kangjie Chen, Guowen Xu, Han Qiu, Shangwei Guo, and Tianwei Zhang. 2024. Protecting Confidential Virtual Machines from Hardware Performance Counter Side Channels. In *54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2024, Brisbane, Australia, June 24-27, 2024*. IEEE, 195–208. doi:10.1109/DSN58291.2024.00031

[15] Elke De Mulder, Michael Hutter, Mark E. Marson, and Peter Pearson. 2013. Using Bleichenbacher"s Solution to the Hidden Number Problem to Attack Nonce Leaks in 384-Bit ECDSA. In *Cryptographic Hardware and Embedded Systems - CHES 2013 - 15th International Workshop, Santa Barbara, CA, USA, August 20-23, 2013. Proceedings (Lecture Notes in Computer Science, Vol. 8086)*, Guido Bertoni and Jean-Sébastien Coron (Eds.). Springer, 435–452. doi:10.1007/978-3-642-40349-1_25

[16] Phong Q. Nguyen and Igor E. Shparlinski. 2003. The Insecurity of the Elliptic Curve Digital Signature Algorithm with Partially Known Nonces. *Des. Codes Cryptogr.* 30, 2 (2003), 201–217. doi:10.1023/A:1025436905711

[17] Fabian Rauscher, Luca Wilke, Hannes Weissteiner, Thomas Eisenbarth, and Daniel Gruss. 2025. TDXploit: Novel Techniques for Single-Stepping and Cache Attacks on Intel TDX. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 1207–1222. https://www.usenix.org/conference/usenixsecurity25/presentation/rauscher

[18] Benedict Schlüter, Supraja Sridhara, Andrin Bertschi, and Shweta Shinde. 2024. WeSee: Using Malicious #VC Interrupts to Break AMD SEV-SNP. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 4220–4238. doi:10.1109/SP54263.2024.00262

[19] Benedict Schlüter, Supraja Sridhara, Mark Kuhne, Andrin Bertschi, and Shweta Shinde. 2024. HECKLER: Breaking Confidential VMs with Malicious Interrupts. In *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association. https://www.usenix.org/conference/usenixsecurity24/presentation/schl%C3%BCter

[20] Matt Spisak. 2016. Hardware-Assisted Rootkits: Abusing Performance Counters on the ARM and x86 Architectures. In *10th USENIX Workshop on Offensive Technologies, WOOT 16, Austin, TX, USA, August 8-9, 2016*, Natalie Silvanovich and Patrick Traynor (Eds.). USENIX Association. https://www.usenix.org/conference/woot16/workshop-program/presentation/spisak

[21] Sebastian Vogl and Claudia Eckert. 2012. Using Hardware Performance Events for Instruction-Level Monitoring on the x86 Architecture. In *Proceedings of EuroSec'12, 5th European Workshop on System Security*. ACM Press.

[22] Samuel Weiser, David Schrammel, Lukas Bodner, and Raphael Spreitzer. 2020. Big Numbers - Big Troubles: Systematically Analyzing Nonce Leakage in (EC)DSA Implementations. In *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, Srdjan Capkun and Franziska Roesner (Eds.). USENIX Association, 1767–1784. https://www.usenix.org/conference/usenixsecurity20/presentation/weiser

[23] Hannes Weissteiner, Fabian Rauscher, Robin Leander Schröder, Jonas Juffinger, Stefan Gast, Jan Wichelmann, Thomas Eisenbarth, Daniel Gruss, and Thomas Eisenbarth. 2025. TEEcorrelate: An Information-Preserving Defense against Performance-Counter Attacks on TEEs. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 2481–2498. https://www.usenix.org/conference/usenixsecurity25/presentation/weissteiner

[24] Luca Wilke, Florian Sieck, and Thomas Eisenbarth. 2024. TDXdown: Single-Stepping and Instruction Counting Attacks against Intel TDX. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM, 79–93. doi:10.1145/3658644.3690230

[25] Luca Wilke, Jan Wichelmann, Anja Rabich, and Thomas Eisenbarth. 2024. SEV-Step A Single-Stepping Framework for AMD-SEV. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2024, 1 (2024), 180–206. doi:10.46586/TCHES.V2024.I1.180-206

A Open Science

We open-source our implementation, configurations, and experiments as a git repository. The implementation includes our kernel module as well as the experiments and our modifications to the TinyCrypt library.

B Additional Experiments

Instruction	AVG	MIN	MAX	Latency
`ADD reg, reg`	2043	1025	6689	1
`DIV reg`	105	53	341	9 - 19
`DIV mem`	105	53	332	10 - 22
`DIV mem*`	64	10	297	10 - 22
`LSL reg`	19	10	60	102 - 106
`IMUL reg`	682	341	2229	3 - 4

Table 1. The table shows the average number of executed instructions in the *test* slide of the experiment in Listing 1 caused by skid. The asterisk in row 4 indicates that the experiment was modified to flush the memory address `mem` at the beginning of each loop iteration. For all tested instructions, we measured a relatively constant 2039 ± 4 `ADD` instructions in the *static* slide.